

APACHE STORM

**DISTRIBUTED, FAULT-TOLERANCE
REAL-TIME COMPUTATION**

**CSCI 109 ALESSANDRA MILEO
ALESSANDRA.MILEO@DCU.IE**

Develop a Storm App

Basics

Develop a Storm App

1. Create a Spout: you will most likely use an existing spout
 - Kafka/Redis/... Spouts
2. Create a Bolt (focus on JVM languages)
 - Implement the `IRichBolt` or `IBasicBolt` interfaces
 - Extend the `BaseRichBolt` or `BaseBasicBolt` classes
3. Create a topology via `TopologyBuilder`
4. Submit a topology
 - locally via `StormSubmitter.submitTopology()`
 - to a cluster via `mylocalcluster.submitTopology()`
5. Run (start) topology via the `storm jar` command
6. Check status of topology (via CLI, Web or API)
7. kill the topology via `storm kill` command

Develop a Storm App

1. Create a Spout: you will most likely use an existing spout
 - Kafka/Redis/... Spouts
2. Create a Bolt (focus on JVM languages)
 - Implement the `IRichBolt` or `IBasicBolt` interfaces
 - Extend the `BaseRichBolt` or `BaseBasicBolt` classes
3. Create a topology via `TopologyBuilder`
4. Submit a topology
 - locally via `StormSubmitter.submitTopology()`
 - to a cluster via `mylocalcluster.submitTopology()`
5. Run (start) topology via the `storm jar` command
6. Check status of topology (via CLI, Web or API)
7. kill the topology via `storm kill` command

2. Create a Bolt: Bolt Interface

- `prepare()`
 - Setup function to initialize the bolt and provide an output collector variable
- `execute()`
 - Receives a tuple from the bolt's input and processes it (the heart of the bolt)
- `cleanup()`
 - Tear-down function (when bolt is shut down)
- `declareOutputFields()`
 - Declares tuples emitted by the bolt

Note: tuples can be *emitted* at any point

3. Create a Topology: Wordcount (local)

Construct topology in Java

```
TopologyBuilder builder = new TopologyBuilder();
```

Define a spout,
parallelism of 5
tasks

```
builder.setSpout("sentences", new RandomSentenceSpout(), 5);
```

Split sentences into words,
parallelism of 8 tasks

```
builder.setBolt("split", new SplitSentence(), 8)  
    .shuffleGrouping("sentences");
```

Connect bolt to input

```
builder.setBolt("count", new WordCount(), 12)  
    .fieldsGrouping("split", new Fields("word"));
```

Create a word
count stream

```
StormTopology topology = builder.createTopology();
```

Connect bolt to input

What happens...

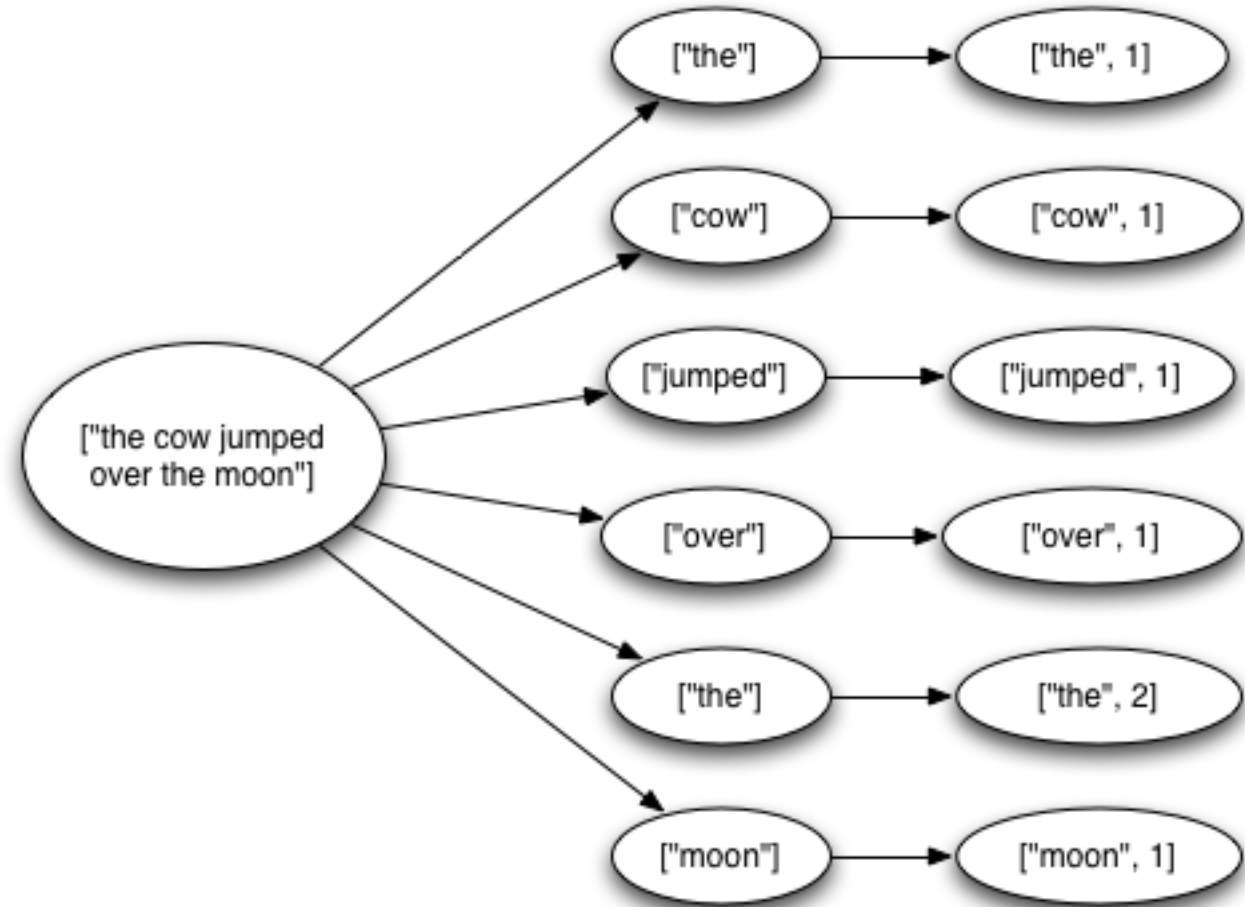
- When a Worker dies?
 - The Supervisor Restarts it
 - If that fails, Nimbus starts it on another machine
- When a Node dies?
 - The heartbeat times out and Nimbus reassigns the tasks
- When Nimbus or a Supervisor dies?
 - Zookeeper manages the state, if monitored, failed Nimbus/Supervisors restart
 - Workers are unaffected by dead supervisors / nimbus
- If Nimbus stays dead
 - The Processing will complete, but no new topology will be assigned
 - Failed tasks will not be reassigned (as this is Nimbus task)

Reliability

- Storm guarantees that every spout tuple will be fully processed by the topology
- One Tuple could theoretically create thousands (e.g. a tokenised string)
 - How to tell if it's been fully processed?
- Maintain a tuple processing tree (message tree)
 - As tuples are acknowledged, mark them in the tree
 - Tuples can time out or be explicitly failed
- If a Tuple is **anchored** when emitted, the tree is replayed if it fails

Example: word count tuple tree

- A tuple for each word in the sentence
- A tuple for the updated count for each word
- The spout tuple (string) is processed when all tuples in the tree are processed
- After a timeout, the spout tuple is replayed



Levels of Reliability

- At-most-once
 - Tuples are processed in order, and if one is missed it's dropped
- At-least-once
 - Tuples are re-emitted if they fail; can be processed more than once or out of order
- Exactly-once
 - Requires Storm Trident to change the semantics (from stateless to stateful)

Storm Trident

- Minibatch Processing for Storm (higher level abstractions)
 - Allows combination of batch processing with Stream Processing
 - 'Pig for Storm'
 - Possibly lower performance
 - Stateful, exactly-once processing
- (<http://storm.apache.org/documentation/Trident-tutorial.html>)

We have not covered...

- Trident details
- Distributed Remote Procedure Calls (DRPC)
- Storm security features
- Storm on EC2
 - see one-click deploy tool at <https://github.com/nathanmarz/storm-deploy>
- Tuning Storm performances
- Monitoring Storm (e.g. Storm metrics and logfiles, Graphite, ...)
- Testing Topologies (e.g. unit test, built-in testing APIs)
- Serialization (via Kryo, falls back onto Java or need custom serialiser)
- See <http://www.slideshare.net/miguno/apache-storm-09-basic-training-verisign>

In Summary...

- Take the divide-and-conquer mapreduce paradigm one step further
- Consider unbounded streams of tuples
- Topology:
 - Spouts produce streams...
 - ... which are processed by bolts
- Reliable transportation (messaging)
- Nimbus co-ordinates Workers (Supervisors) via Zookeeper

APACHE STORM

**DISTRIBUTED, FAULT-TOLERANCE
REAL-TIME COMPUTATION**

CA675 ALESSANDRA MILEO

ALESSANDRA.MILEO@DCU.IE